Un guide gastronomique

Compétences développées dans le module

- Ecrire des requêtes d'interrogation à une table
  - Projections simples
  - Restrictions
- Ecrire des requêtes en affichant des résultats parlants pour l'utilisateur : renommage des colonnes de la table de résultat, tri des résultats.
- Ecrire des requêtes d'interrogation complexes à plusieurs tables
  - Jointures
- Faire des calculs sur une base de données
  - Avec des fonctions d'agrégation sans partition
  - Avec partition
- Ecrire des requêtes reposant sur des sous-requêtes
  - IN et NOT IN

Le syndicat d'initiative d'une grande ville a décidé de constituer un guide gastronomique des restaurants de la cité.
Ce guide doit renseigner les résidents, habitants et touristes sur les différents plats, et tout particulièrement les spécialités, que l'on peut déguster en ville et sur les vins mis en vente dans les restaurants.

La base de données permet d'avoir :

- Les **restaurants** de la ville, enregistrés avec un code d'identification, un nom, une adresse, les jours d'ouverture, leurs horaires d'ouverture à midi et le soir, la liste des plats proposés à la carte et la liste des vins proposés.
- Les **plats** que l'on peut déguster en ville. Pour chaque plat, on doit connaître son nom, sa place dans le repas (entrée, dessert, ...), la liste des prix dans les restaurants qui le proposent à leur carte, le nom éventuel du vin idéal (d'après un grand gastronome) et une couleur de vin recommandée permettant de choisir un vin adapté dans le cas où l'on ne peut prendre le vin idéal. Cette couleur peut être différente de la couleur du vin idéal.
- Les **vins** que l'on peut trouver en ville avec, pour chacun d'eux, des informations sur leur nom, la date de récolte du raisin, la région d'origine, la couleur, et la liste des prix dans les restaurants qui le proposent.

Voici le schéma relationnel qui a été proposé pour cette base :

Restaurants (code, nom, ville)
Vins (code, cru, millesime, region, couleur)
Plats (code, nom, place, couleurVinRdee, #codeVinIdeal)
ServirVin (#codeRestau, #codeVin, prixVin)
ServirPlat (#codeRestau, #codePlat, prixPlat, specialite)
Ouverture (#codeRestau, libJour, horaireOuverture, horaireFermeture)

Ecrire les requêtes SQL qui permettent de répondre aux questions ci-dessous.

TD2 et TP3 : Projections et Restrictions

1) Projections

Q1. Quels sont les noms et villes de tous les restaurants ?
Q2. Quels sont les vins proposés (cru) ?
Q3. Quels sont les vins proposés (cru et millésime) ?

2) Projections et restrictions

Q4. Quels sont les vins rouges du catalogue ?
Q5. Quels sont les vins blancs du millésime 2018 ?
Q6. Quels sont les restaurants de Bayonne et de Biarritz ?
Q7. Quels sont les plats servis en entrée qu'il est conseillé de consommer avec un vin Blanc ?
Q8. Quels sont les codes des plats dont le prix est compris entre 20 et 30 euros ?
Q9. Quels sont les codes des plats qui sont une spécialité du restaurant de code 3 et qui ont un prix inférieur à 15 euros ?
Q10. Quels sont les entrées et les plats pour lesquels on recommande du vin blanc ?
Q11. Quels sont les entrées pour lesquelles on recommande du vin blanc et les plats pour lesquels on recommande du vin rouge ?

Point cours complémentaire pour le TP

Comment améliorer la lisibilité des résultats d'une requête ?

a) Nom des colonnes

Par défaut, une requête renvoie **comme résultat** une table affichant les noms des colonnes indiquées dans la projection.
Il est possible de modifier ces noms dans la table renvoyée en résultat, afin de les rendre plus explicites.
On utilise pour cela des **ALIAS de colonne** dans la clause SELECT :

```
SELECT nomColonne AS nouveauNom
```

b) Tri des résultats

Les résultats d'une requête sont retournés dans l'ordre chronologique d'insertion des enregistrements dans la base de données.
Cependant, pour améliorer la lisibilité, il pourrait être intéressant de trier ces résultats sur une ou plusieurs colonnes.

On utilise pour cela la clause ORDER BY, placée à la fin de la requête.

```
SELECT
FROM
[WHERE]
ORDER BY nomColonne1 {ASC|DESC}, …, nomColonneN {ASC|DESC};
```

ASC : croissant
DESC : décroissant